



how can i seperate customer login from business login? what workflow do you suggets

Separate customer and business logins in your FoodLoop Flask app by using distinct routes (e.g., `/login` for customers, `/business` for businesses) with role-based checks on the shared `User` model. This prevents cross-login attempts, as producers are blocked from customer login and vice versa.^{[1] [2]}

Separation Strategy

Use separate routes to handle authentication flows tailored to each role (`CUSTOMER` vs `PRODUCER`). In each route's email step, query the DB by email and enforce role matching before showing the password form—if the role doesn't match, flash a message like "Please use Business Login" and stay on the email step.^{[1] [3] [4]}

- Customer route (`/login`): Only proceed if `user.role == "CUSTOMER"` or user doesn't exist (then offer signup).
- Business route (`/business`): Only proceed if `user.role == "PRODUCER"` or new user (signup creates `PRODUCER` role and `ProducerProfile`).
- Post-login: Redirect customers to `/account`, businesses to `/business_account` dashboard.^{[5] [6]}

This leverages Flask-Login's single session per role, ensuring no overlap since mismatched roles can't authenticate.^[2]

Suggested Workflow

Adopt an email-first, multi-step flow in a single template per route (e.g., `login.html` and `business.html`) for simplicity—no JS needed.^{[7] [8]}

1. **Landing/Email Step:** Show email form only. On submit, check DB.
2. **Branch on Existence/Role:**

Condition	Action
Email exists, correct role	Next: Password form.
Email exists, wrong role	Message + stay on email (e.g., "Use the other login").
Email new	Next: Signup form with role-specific fields (e.g., business name/address for producers).

3. Password/Signup Success: Create/update User, role-specific profile (CustomerProfile or ProducerProfile), login, and redirect to dashboard.

Example route snippet for /login (adapt for /business by swapping role):

```
@app.route("/login", methods=["GET", "POST"])
def login():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            user = User.query.filter_by(email=email).first()
            if user and user.role != "CUSTOMER":
                flash("Please use Business Login.")
                return render_template("login.html", step="email", email=email, form=form)
            return redirect(url_for("login", step="password", email=email))
        return render_template("login.html", step="email", email=email, form=form)

    # Similar for password/signup steps...
```

This matches your existing multi-step pattern and course constraints.[\[8\]](#) [\[1\]](#)

✱

i need an if block for each step right?

Yes, structure your Flask login routes with an if-elif chain (or separate if blocks) for each step to handle the multi-step flow cleanly in one route. This keeps logic centralized, uses URL params like ?step=password for navigation, and matches your existing FoodLoop app pattern.[\[19\]](#) [\[20\]](#) [\[21\]](#)

Route Structure

Define `step = request.args.get("step", "email")` at the top, then use sequential `if step == "email": ... elif step == "password": ... elif step == "signup": ...` blocks. Each block renders the matching form/template snippet and handles form submits by redirecting to the next step (e.g., `redirect(url_for("login", step="password", email=email))`). A final `else:` catches invalid steps and redirects to "email".[\[21\]](#) [\[19\]](#)

Example for /login

```
@app.route("/login", methods=["GET", "POST"])
def login():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            user = User.query.filter_by(email=email).first()
```

```

        if user and user.role != "CUSTOMER":
            flash("Please use Business Login.")
            return render_template("login.html", step="email", email=email, form=form)
        return redirect(url_for("login", step="password", email=email))
    return render_template("login.html", step="email", email=email, form=form)

elif step == "password":
    form = PasswordOnlyLoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(email=email, role="CUSTOMER").first()
        if user and check_password_hash(user.password_hash, form.password.data):
            login_user(user)
            return redirect(url_for("account"))
        flash("Invalid password.")
    return render_template("login.html", step="password", email=email, form=form)

elif step == "signup":
    form = SignupForm()
    if form.validate_on_submit():
        user = User(email=email, role="CUSTOMER", ...) # Hash password, etc.
        db.session.add(user)
        db.session.commit()
        login_user(user)
        return redirect(url_for("account"))
    return render_template("login.html", step="signup", email=email, form=form)

return redirect(url_for("login")) # Invalid step fallback

```

Mirror this for /business (swap role to "PRODUCER", use SignupFormProducers). In login.html, use {% if step == "email" %} ... {% elif step == "password" %} ... {% endif %} to show the right form. [\[20\]](#) [\[22\]](#)

**

1. projects.web_app_authentication
2. <https://www.perplexity.ai/search/184c469a-0f76-416c-ab1b-c9e14c42ec65>
3. <https://stackoverflow.com/questions/15871391/implementing-flask-login-with-multiple-user-classes>
4. <https://www.perplexity.ai/search/da8e4fb6-dded-4282-a492-54f7d2458b01>
5. <https://qwiet.ai/appsec-resources/securing-your-flask-applications-essential-extensions-and-best-practices/>
6. <https://www.perplexity.ai/search/c5f3c725-d8e5-40ed-a66d-81d8dc7fa5a3>
7. <https://www.perplexity.ai/search/b6668764-d544-405a-9786-580eda2be13e>
8. <https://www.perplexity.ai/search/7fe58c43-ad6f-46cb-b714-c6d3c761de9b>
9. preferences.architecture_db
10. <https://community.make.com/t/best-practice-single-vs-multiple-make-com-accounts-for-client-management/57400>
11. https://www.reddit.com/r/softwarearchitecture/comments/ne01eg/why_do_many_applications_sometimes_have_separate/
12. <https://www.youtube.com/watch?v=NSrhsNM2eqE>

13. <https://community.shopify.com/t/multiple-users-profiles-logins-under-one-shopify-account/49811>
14. <https://community.auth0.com/t/organization-login-experience-workflow-ambiguity/195350>
15. <https://stackoverflow.com/questions/69205363/separate-login-sessions-for-multiple-subdomains-in-a-single-flask-app>
16. <https://forum.bubble.io/t/can-a-workflow-log-in-as-a-specific-user/48726>
17. <https://flask-user.readthedocs.io/en/latest/authorization.html>
18. <https://www.perplexity.ai/search/46255b9b-78b1-47bc-b774-fadc70b22cea>
19. <https://www.perplexity.ai/search/14b44f51-d87a-48e1-bcb1-8e98da8e014a>
20. <https://www.perplexity.ai/search/3e827447-abe1-4473-9954-aba9823046bd>
21. <https://www.perplexity.ai/search/cc36bb8e-be30-4bc0-a228-00955f1e6851>
22. <https://www.perplexity.ai/search/46255b9b-78b1-47bc-b774-fadc70b22cea>